Single crystal adventures: Wombat, the Eulerian cradle, and DMCpy

Siobhan, April 2025

Contents

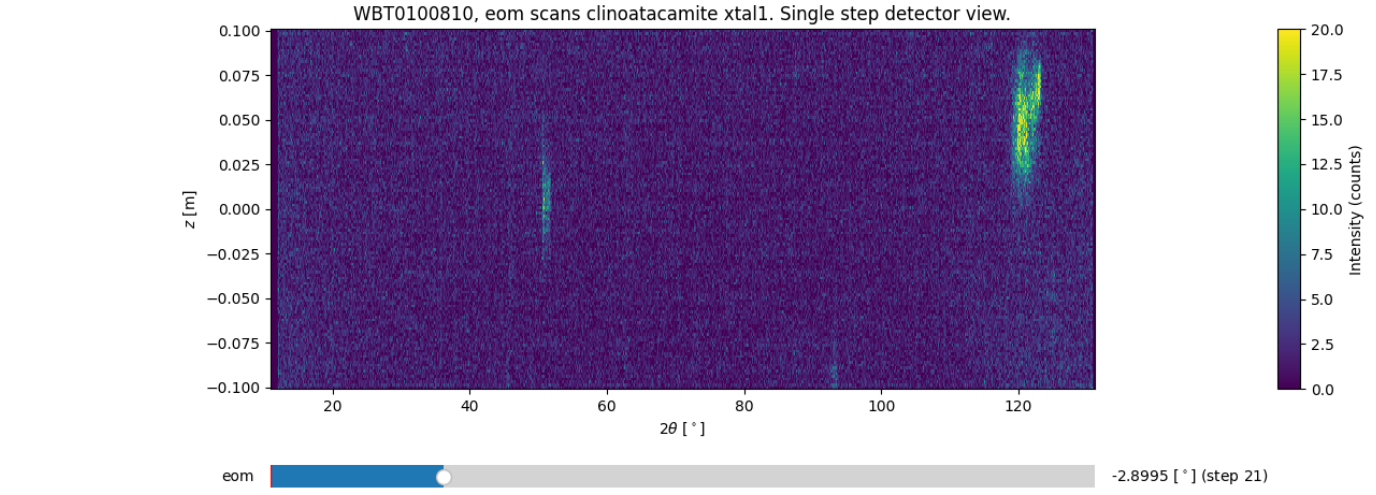
Single crystal examples from 2025

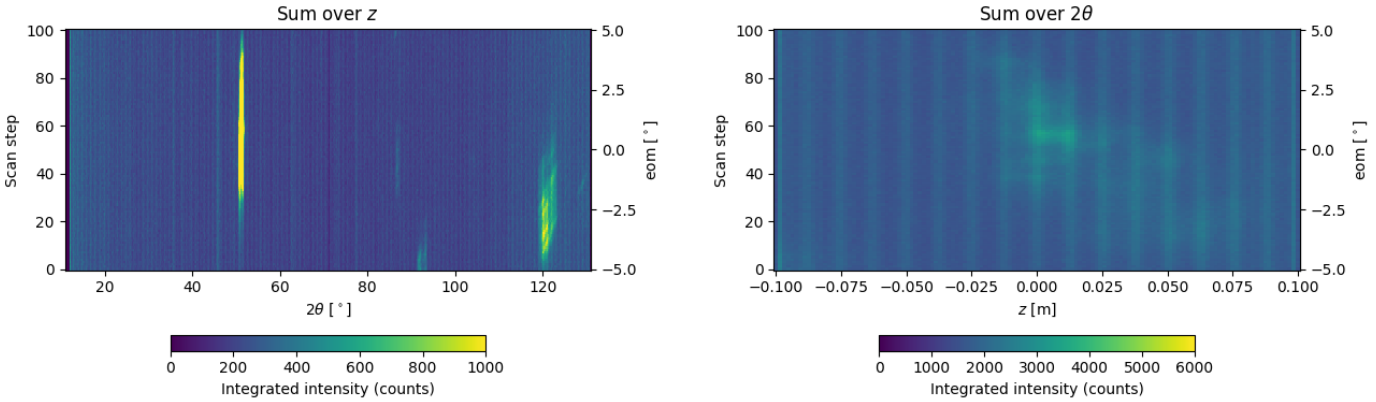
Tips for data collection

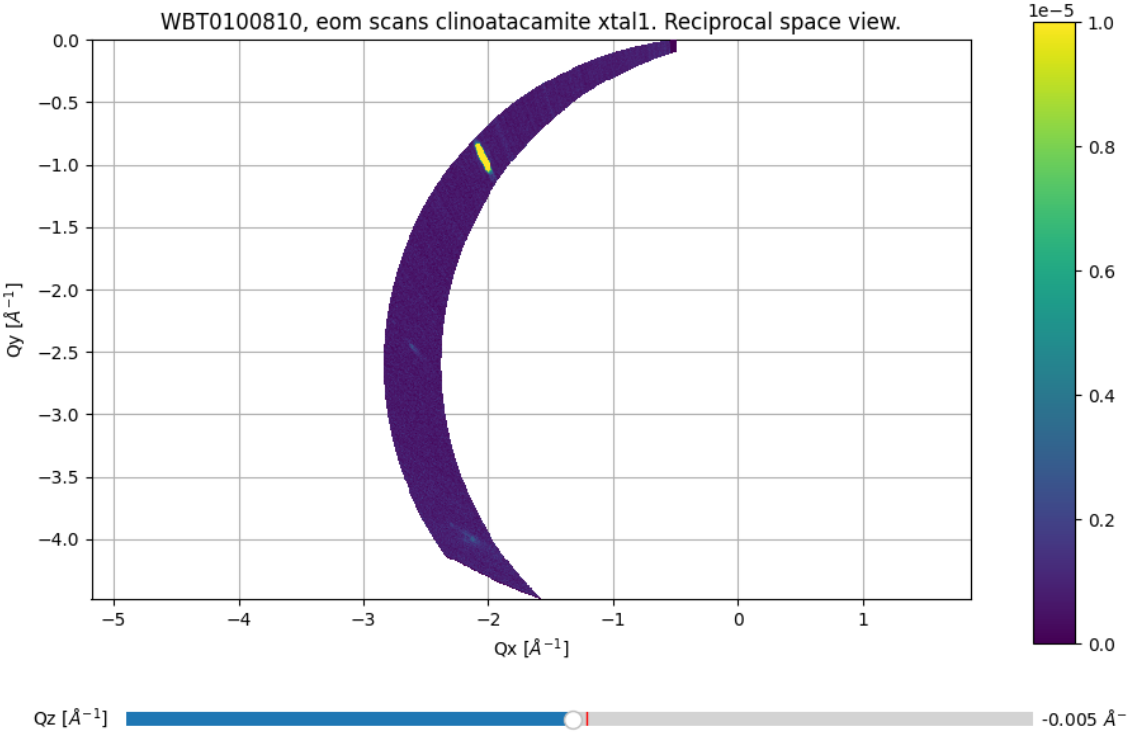
- All HDF should have the same number of steps if you want to combine them
- 0.1° steps is good for Wombat (smaller is pointless)
- Fewer acquisitions covering a broader range > more acquisitions over a smaller range about random angles

Kirrily's clinoatacamite

- This was the first experiment I did using the Eulerian cradle on Wombat
- Was very much a test run and spent some time testing/fixing limits with Frank and Helen
- Did not have an optimal strategy for data collection
    - Initially was doing 0.01° steps in eom, which was ridiculous
    - Then did 10° eom scans with 0.1° steps about random echi and epsi angles…
- Sample mounted on Al pin with blu tac (classy)
- Crystal is small and twinned – didn't cover enough of *Q* space to find orientation or seriously identify peaks

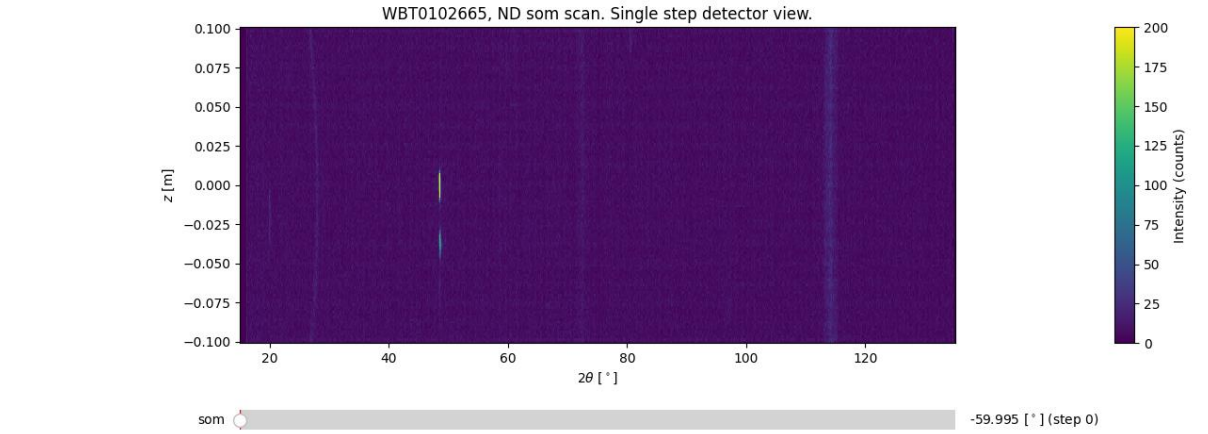

WBT0100810, eom scans clinoatacamite xtal1. Single step detector view.
z [m]
2θ [˚]
Intensity (counts)
eom
-2.8995 [˚] (step 21)

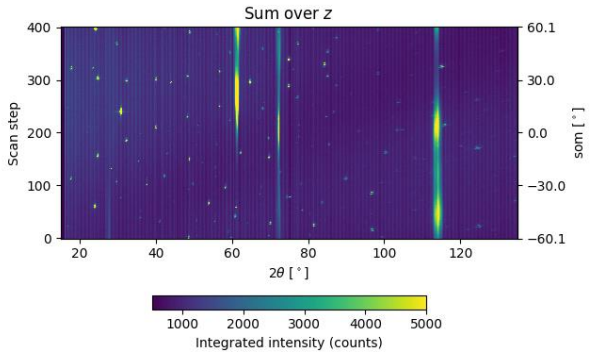

Sum over z
Scan step
2θ [˚]
eom [˚]
Integrated intensity (counts)
Sum over 2θ
z [m]

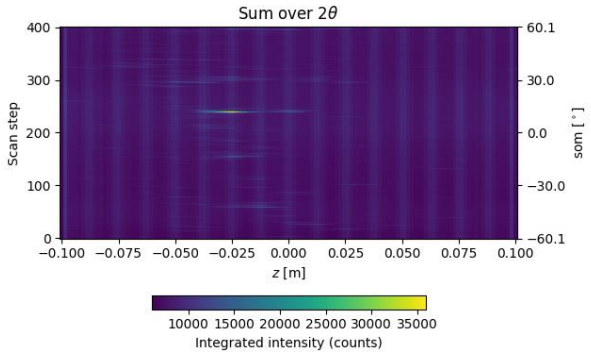

WBT0100810, eom scans clinoatacamite xtal1. Reciprocal space view.
Qy [Å⁻¹]
Qx [Å⁻¹]
1e−5
Qz [Å⁻¹]
-0.005 Å⁻¹

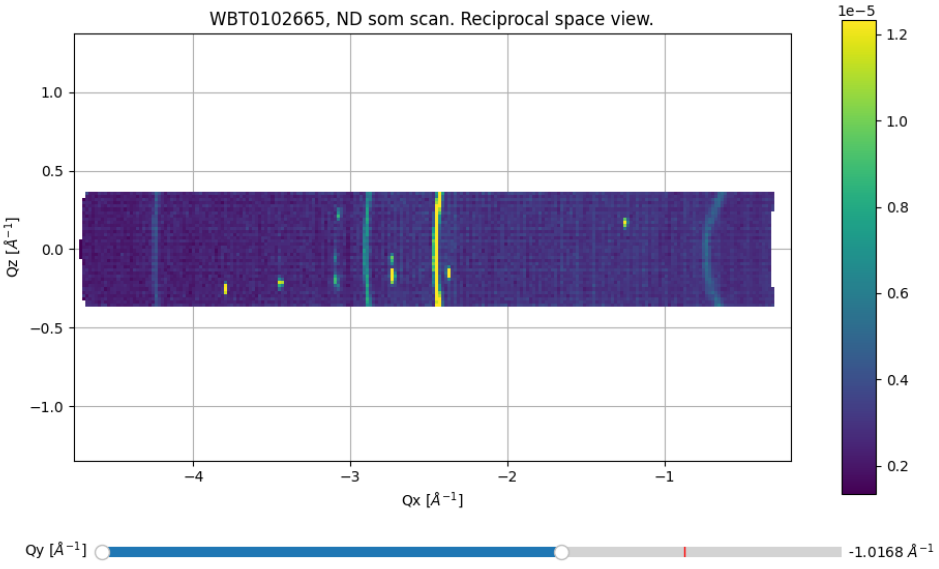
Alex's mysterious ND crystal

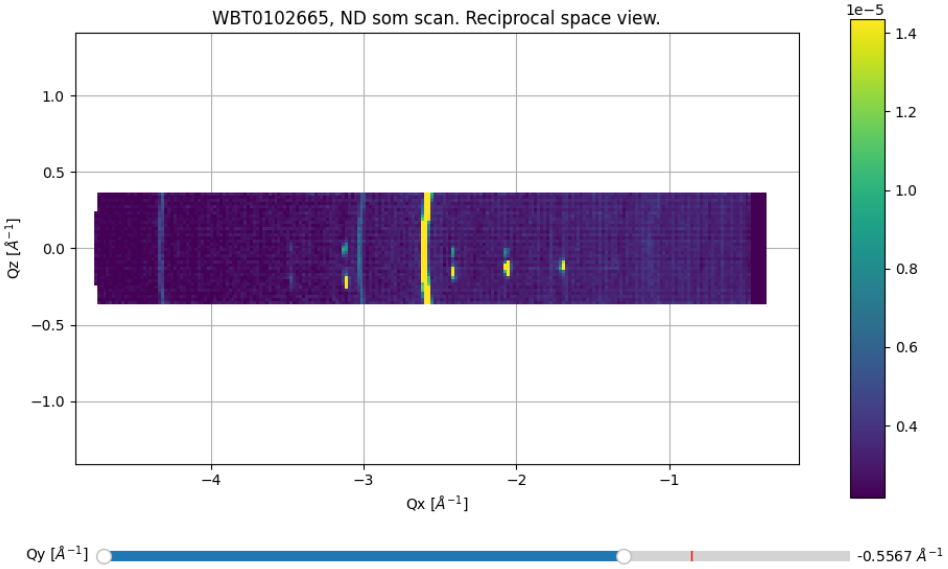
- No Eulerian cradle, just sample on Al plate on stick on Wombat sample stage (prominent Al rings)
- Did a long som scan that covered 120° in 0.3° steps
- Smallish crystal is a little bit twinned (see Qx-Qz projections below)
- Could probably find orientation of biggest twin?



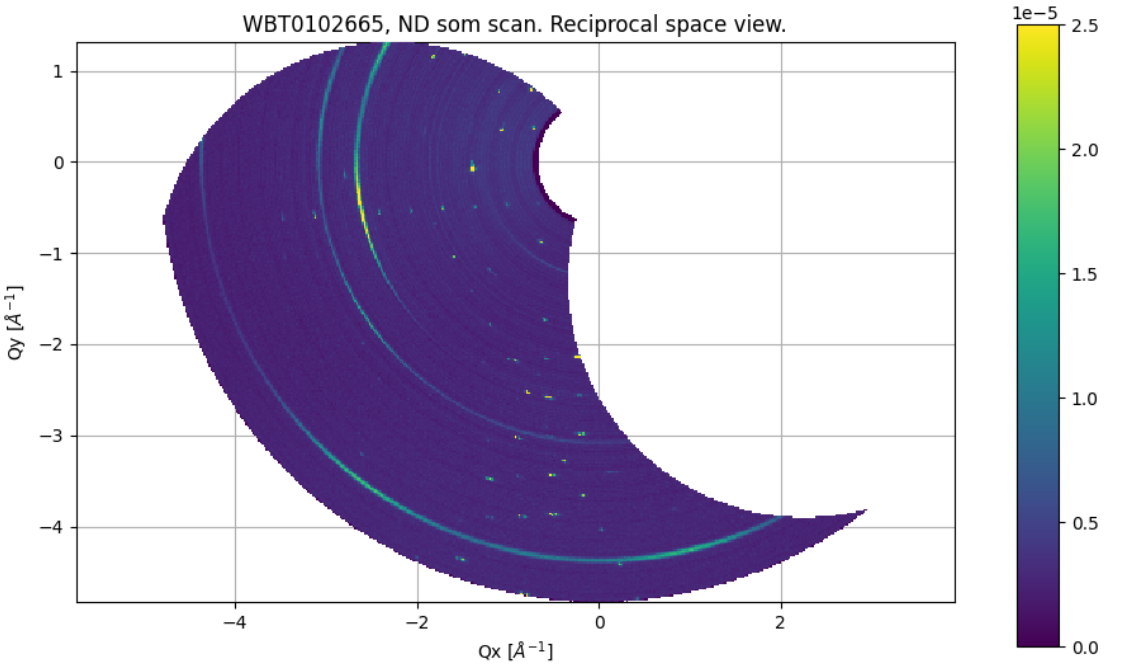







WBT0102665, ND som scan. Reciprocal space view.
1e−5
1.4
1.2
1.0
0.8
0.6
0.4
1.0
0.5
0.0
−0.5
−1.0
Qz [Å^{-1}]
−4
−3
−2
−1
Qx [Å^{-1}]

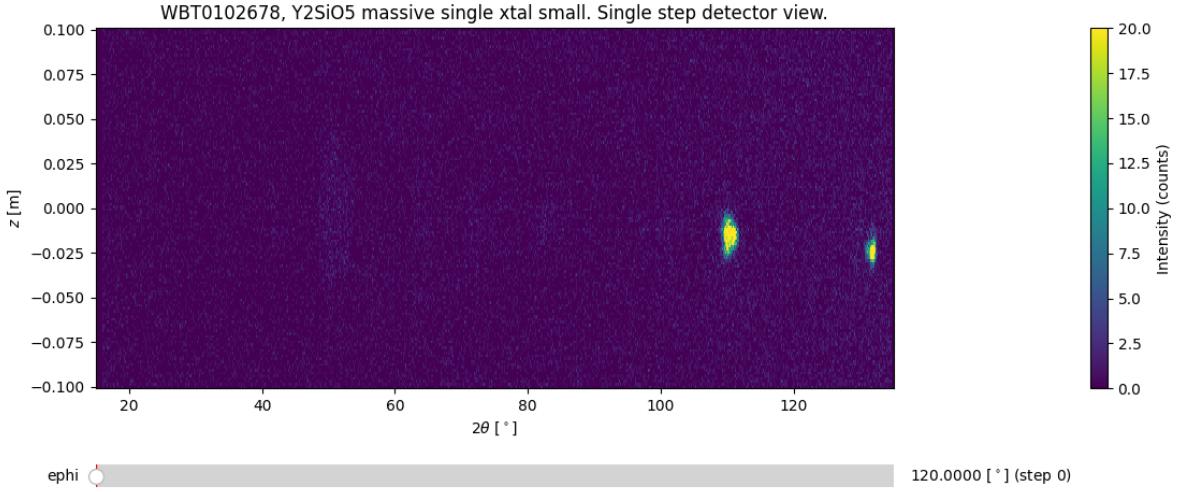
Qy [Å^{-1}]
-0.5567 Å^{-1}

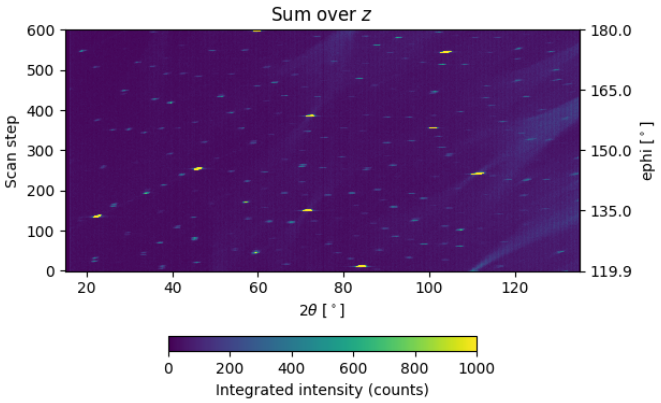

WBT0102665, ND som scan. Reciprocal space view.
1e−5
2.5
2.0
1.5
1.0
0.5
0.0
1
0
−1
−2
−3
−4
Qy [Å^{-1}]
−4
−2
0
2
Qx [Å^{-1}]

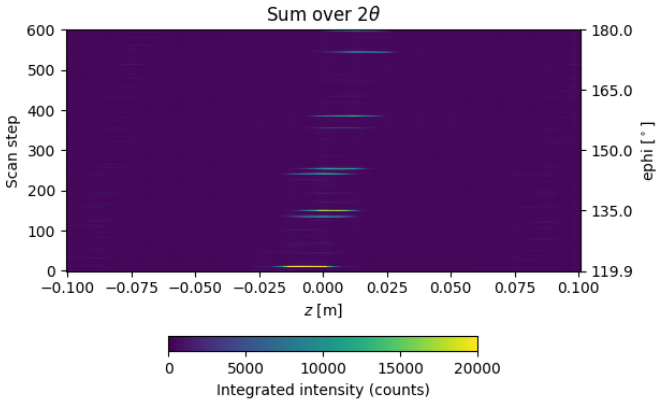

Qz [Å^{-1}]
-0.0101 Å^{-1}

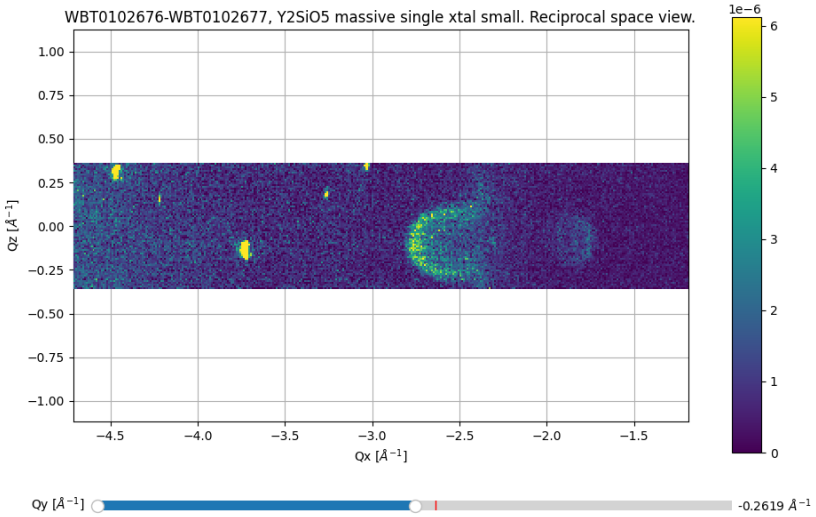
Federico's ginormous Y_2SiO_5 crystal

- Ginormous crystal of Y_2SiO_5 (about $2cm^3$) mounted on Al pin secured with Al tape
- We knew that crystal was mounted with scattering plane ~ *h0l*
- So big we even saw phonons… on Wombat
- Mounted crystal on Eulerian cradle
- 3 x collections, each spanning 60° in epsi, 600 x 0.1° steps (#102676-677), 601 x 0.1° steps (#102678), counting for 30 sec at each step!
- Later I figured out having all HDF with the same number of steps is good… oops
- I figured out the UB matrix for this crystal: really good Q space coverage, clear peaks to index, very nice single crystal, and a bit of prior knowledge!



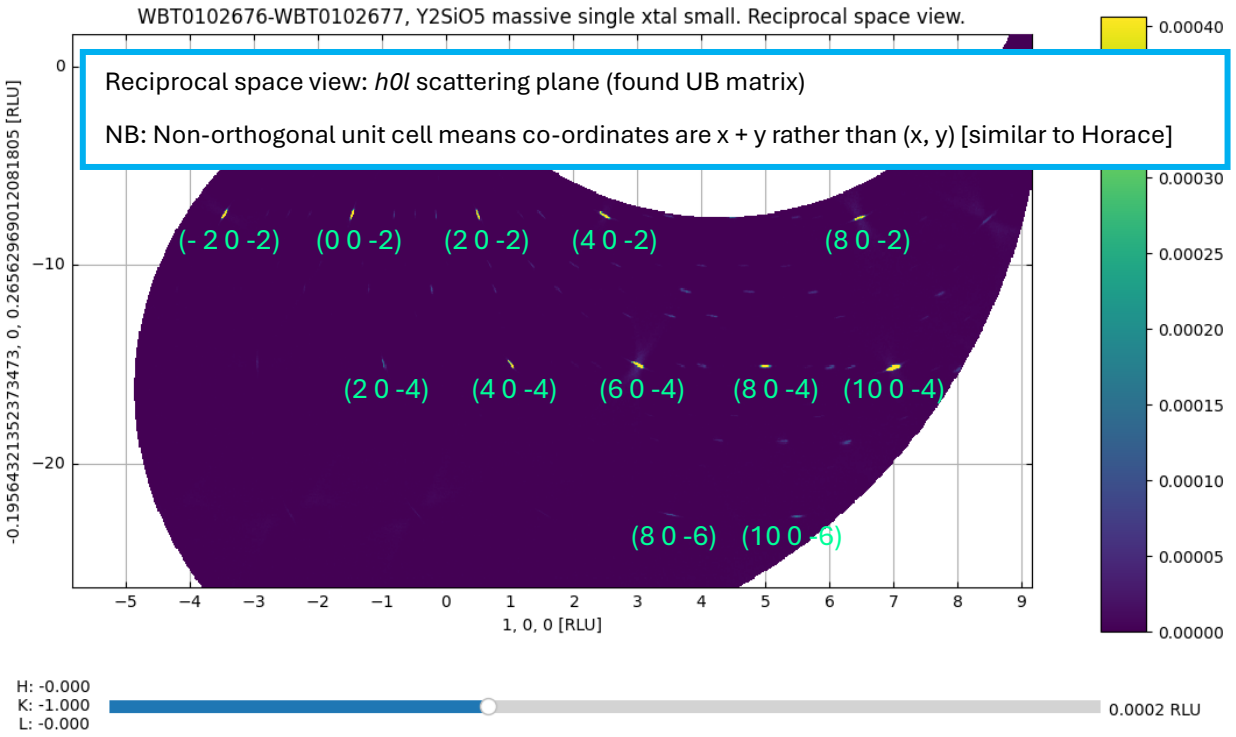


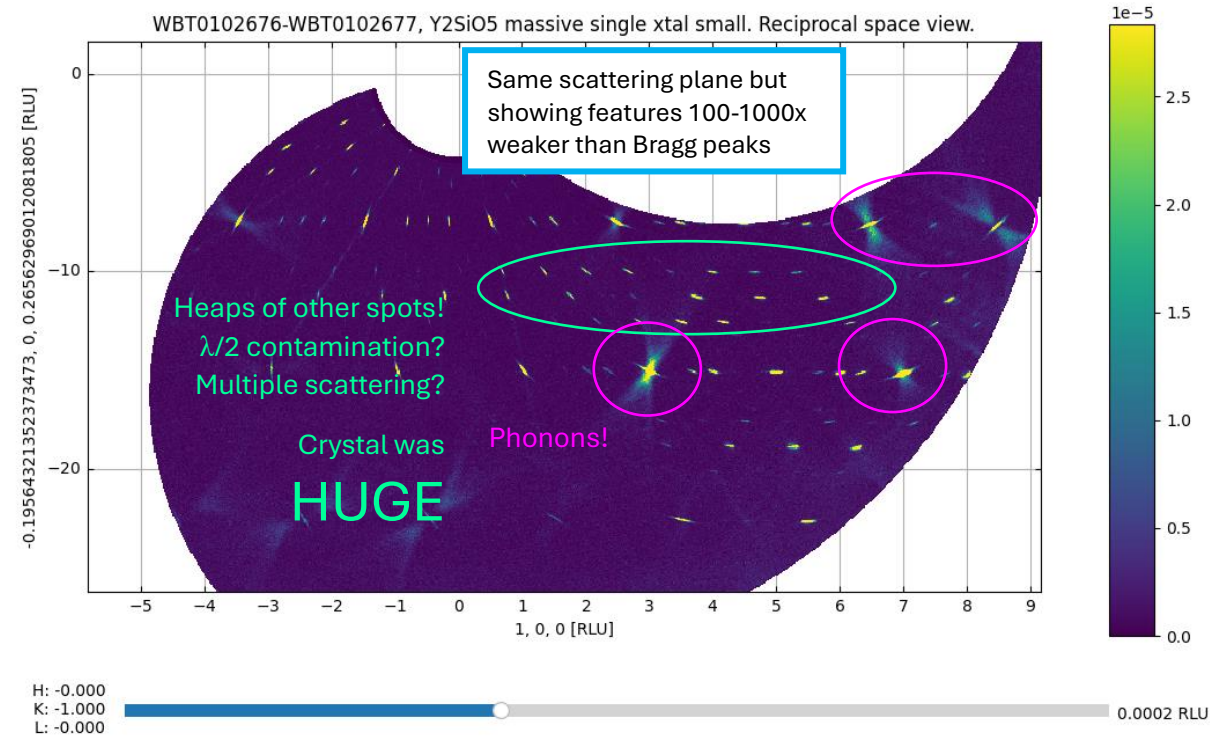




Interesting phonons seen in out-of-plane reciprocal space view!

The peak at (Qx, Qy, Qz) ~ (-3.7, -0.26, -0.18) is a Bragg peak. Not sure what the four smaller peaks are at the top of the detector, though they are ~300x weaker than the Bragg peak.


WBT0102676-WBT0102677, Y2SiO5 massive single xtal small. Reciprocal space view.
Reciprocal space view: h0l scattering plane (found UB matrix)
NB: Non-orthogonal unit cell means co-ordinates are x + y rather than (x, y) [similar to Horace]
(- 2 0 -2)
(0 0 -2)
(2 0 -2)
(4 0 -2)
(8 0 -2)
(2 0 -4)
(4 0 -4)
(6 0 -4)
(8 0 -4)
(10 0 -4)
(8 0 -6)
(10 0 -6)
-0.19564321352373473, 0, 0.26562969012081805 [RLU]
1, 0, 0 [RLU]
H: -0.000
K: -1.000
L: -0.000
0.0002 RLU


WBT0102676-WBT0102677, Y2SiO5 massive single xtal small. Reciprocal space view.
Same scattering plane but showing features 100-1000x weaker than Bragg peaks
Heaps of other spots!
λ/2 contamination?
Multiple scattering?
Crystal was
HUGE
Phonons!
-0.19564321352373473, 0, 0.26562969012081805 [RLU]
1, 0, 0 [RLU]
H: -0.000
K: -1.000
L: -0.000
0.0002 RLU

Software: DMCpy x Wombat

Motivation

- Want a Python script to be able to handle Wombat single crystal data and quickly make projections of data in Qx, Qy, Qz from HDF where steps are of sample rotation angle
- Want to be able to determine rough UB matrix and visualise data in *hkl* (i.e. r.l.u.)
- Ideally this can handle arbitrary Eulerian cradle angles (!!!)
- Don't want to use LAMP
- Int3D implementation still a work in progress – this is best for quantitative analysis (ellipsoid integration of peaks) however
    - learning curve for users (and instrument scientists)
    - development of software done by Nebil Kaptcho at ILL so hard to make quick changes
    - need to get ANSTO IT to install it on ANSTO machines
    - runs on its own separate python installation and fortran compiler thing (intimidating)
- Several packages out there however
    - Some have dependencies that only work on Linux e.g. *hklpy* looks really good https://blueskyproject.io/hklpy/ but ultimately depends on which is impossible to run on Windows (I tried. Other people have also tried. And failed.)
    - Some for x-ray detectors that are point detectors or flat area detectors, or goniometer geometries we don't have e.g. *xrayutilities* (Kriegner et al. J. Appl. Cryst. (2013) 46, 1162-1170) https://pypi.org/project/xrayutilities/
    - I thought about which instruments were most similar to Wombat which led me to …

DMCpy

- DMC is a cold neutron diffractometer at PSI that has an area detector very similar to Wombat (in fact Wombat's new detector will be like DMC's)
- *DMCpy* is a python package for reduction of DMC single crystal and powder data developed at DMC by Jakob Lass, Samuel Harrison Moody, and Øystein Slagtern Fjellvåga. It has been used on the instrument for analysis since 2022. Credit to them for making it freely available and documenting it in several different ways.
- It pretty much does everything we need (except we do powder reduction differently)
- It is available from pip https://pypi.org/project/DMCpy/
- It is described in this pre print https://arxiv.org/abs/2501.08845
- Documentation is here https://dmcpy.readthedocs.io/en/latest/index.html including some tutorials (scripts only, no example data)
- Public Git repo is https://github.com/Jakob-Lass/DMCpy

DMCpy x Wombat

wombatDMCpy

To avoid ambiguity, I changed the name of my local copy of *DMCpy* to *wombatDMCpy*. To make DMCpy work for Wombat, the big changes were:

- Different HDF file structure (hence new WombatFileStructure)
- Different way of organising histogram data
- Different definitions of some angles etc
    - A3 is DMCpy's som
    - -A4 is their 2θ
    - Z is their vertical detector position (angle between sample and vertical detector position is called alpha)

- Different detector geometry and sample rotation options (hence new classes WombatDataFile and WombatDataSet)
    - Wombat can rotate the sample about som, echi, epsi, eom, and the sample environment stick axis msom

Some small tweaks to the functionality of Interactive Viewer (raw detector view) and Viewer 3D (reciprocal space view) have also been made for the Wombat context.

NB: no work has been done to make the powder diffraction functions of DMCpy compatible with Wombat

wombatDMCpy is available at https://github.com/queenofthefairies/Wombat_DMCpy/tree/wombat_WIP (Wombat_WIP branch as original DMCpy package is on master branch currently)

I have been running wombatDMCpy in its own venv in MS Visual Studio code. It should also work in Spyder etc but this might kill the interactivity in the plot windows.

wombatDMCpy example using Y_2SiO_5 data

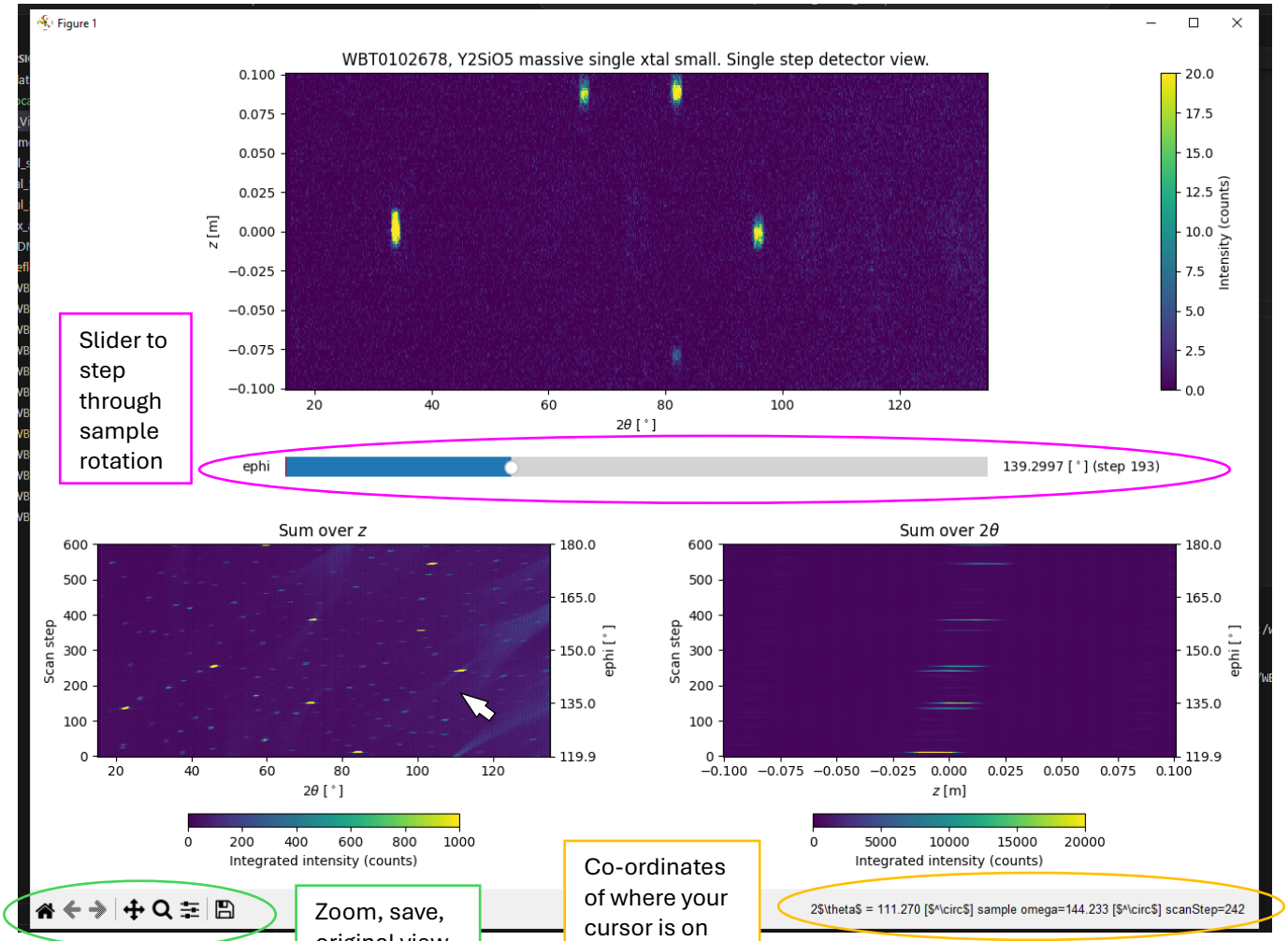
Scripts and test data are available here:
https://github.com/queenofthefairies/Wombat_DMCpy/tree/wombat_WIP/wombat_Y2SiO5_example

Interactive detector view

Here you can look at the raw data, step by step (by moving the scroll bar, in this example, steps in ephi), as well as integrating over the detector vertically (integrating over z) or horizontally (integrating over 2θ). All colour scale limits can be set in the script.

Script: Detector_View_Y2SiO5.py

Relevant DMCpy tutorial:
https://dmcpy.readthedocs.io/en/latest/Tutorials/InteractiveViewer/InteractiveViewer.html



Reciprocal space view in Å^{-1}

Qx-Qy is the scattering plane of the detector. Qz is the out-of-plane component.

If merging HDF together, the HDF files must have the same number of steps!

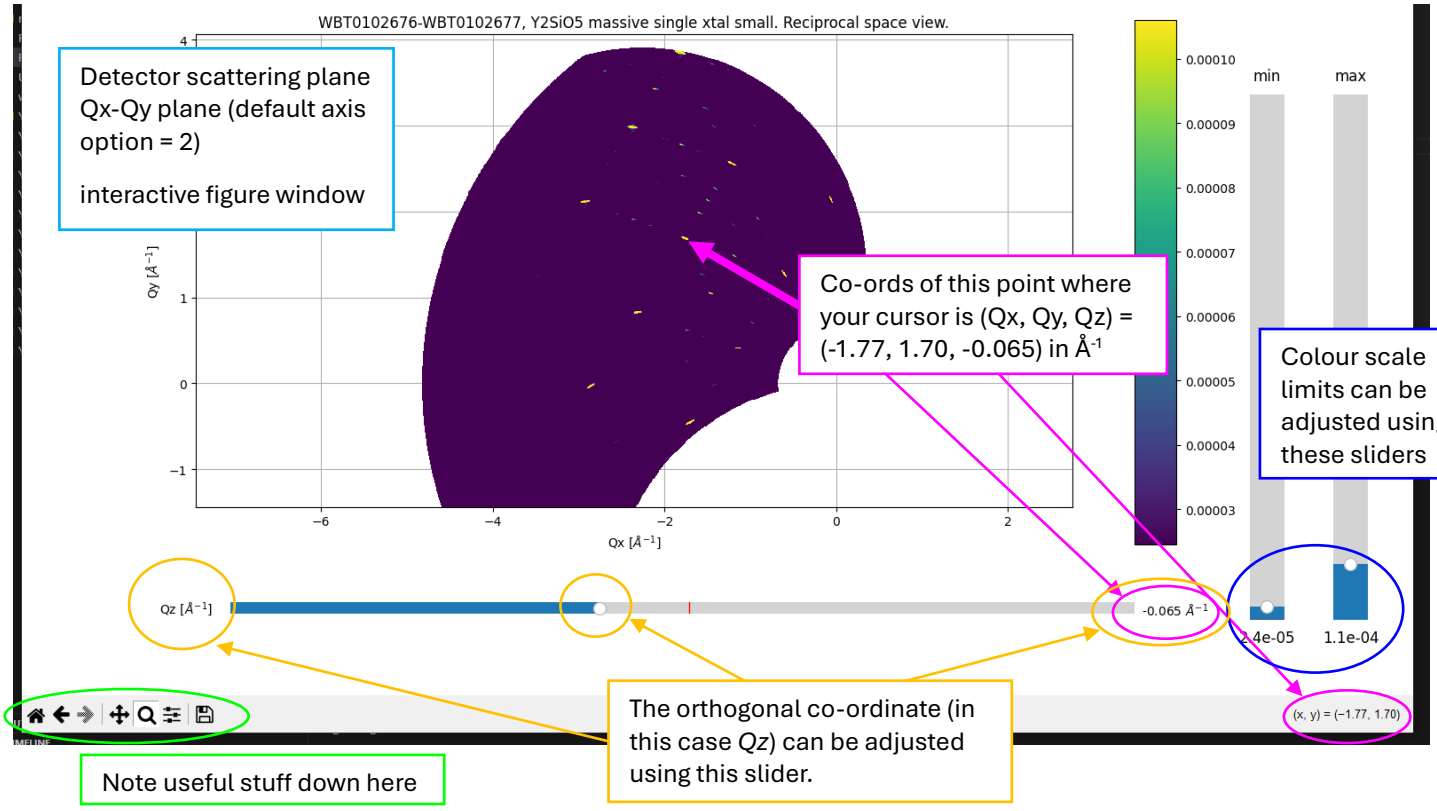
You can select which plane to plot (Qx-Qy, Qx-Qz, Qy-Qz) in the script (or press 0, 1, or 2 when in the interactive plot window, but this is a bit unreliable). The value of the intersection with the remaining axes (Qz, Qy, Qx respectively) is controlled by a slider.

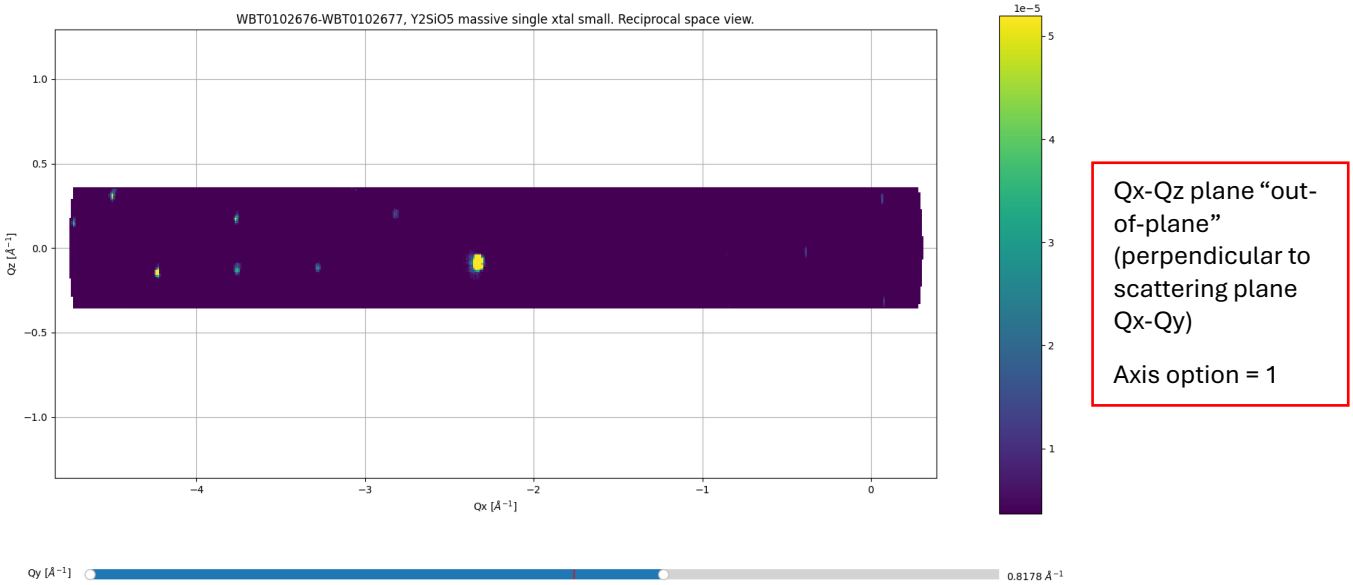
You can adjust the colour scale limits in the interactive pop-up window. You can also save the figure either from within the script or from the interactive window.

Script: Reciprocal_Space_View_invAA_Y2SiO5.py

Relevant DMCpy tutorial:

https://dmcpy.readthedocs.io/en/latest/Tutorials/View3D/Viewer3D.html





UB matrix determination

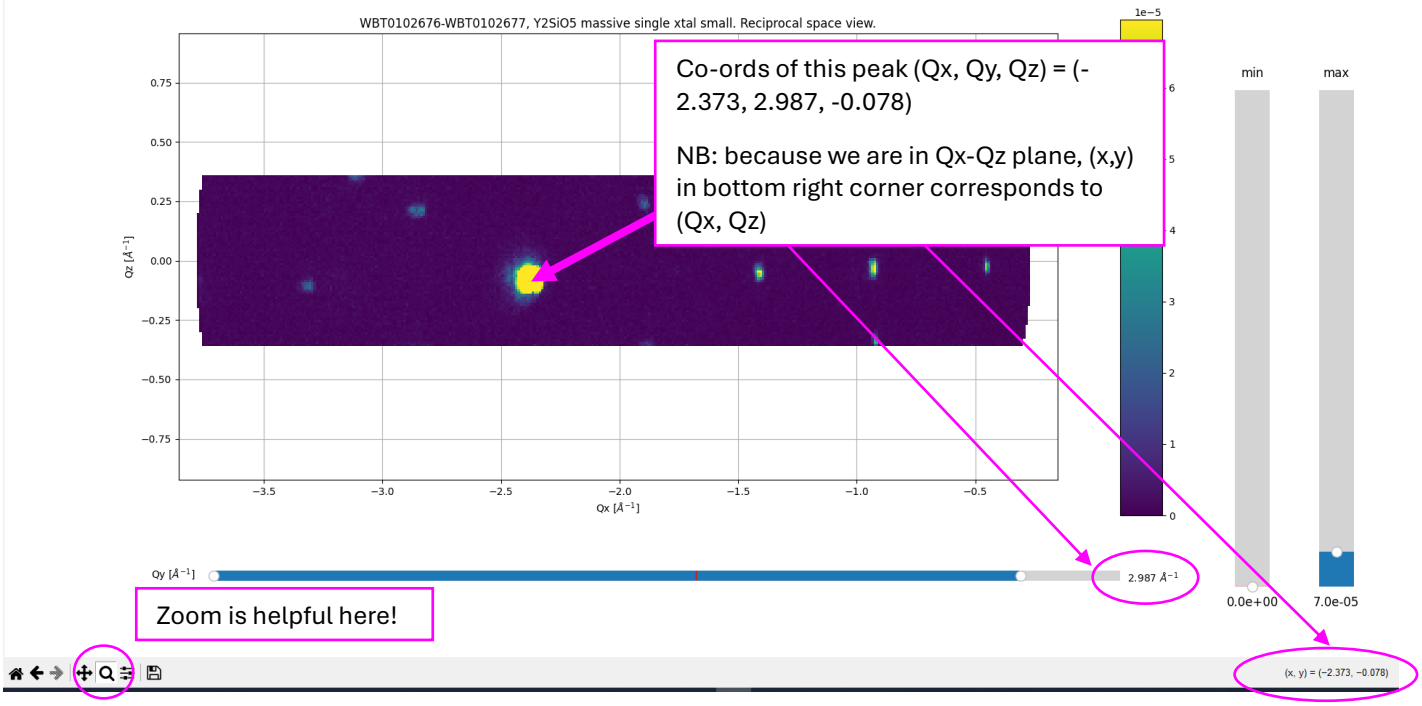
Before calculating a UB matrix, you need to know the unit cell, and it is necessary to index some peaks by hand.

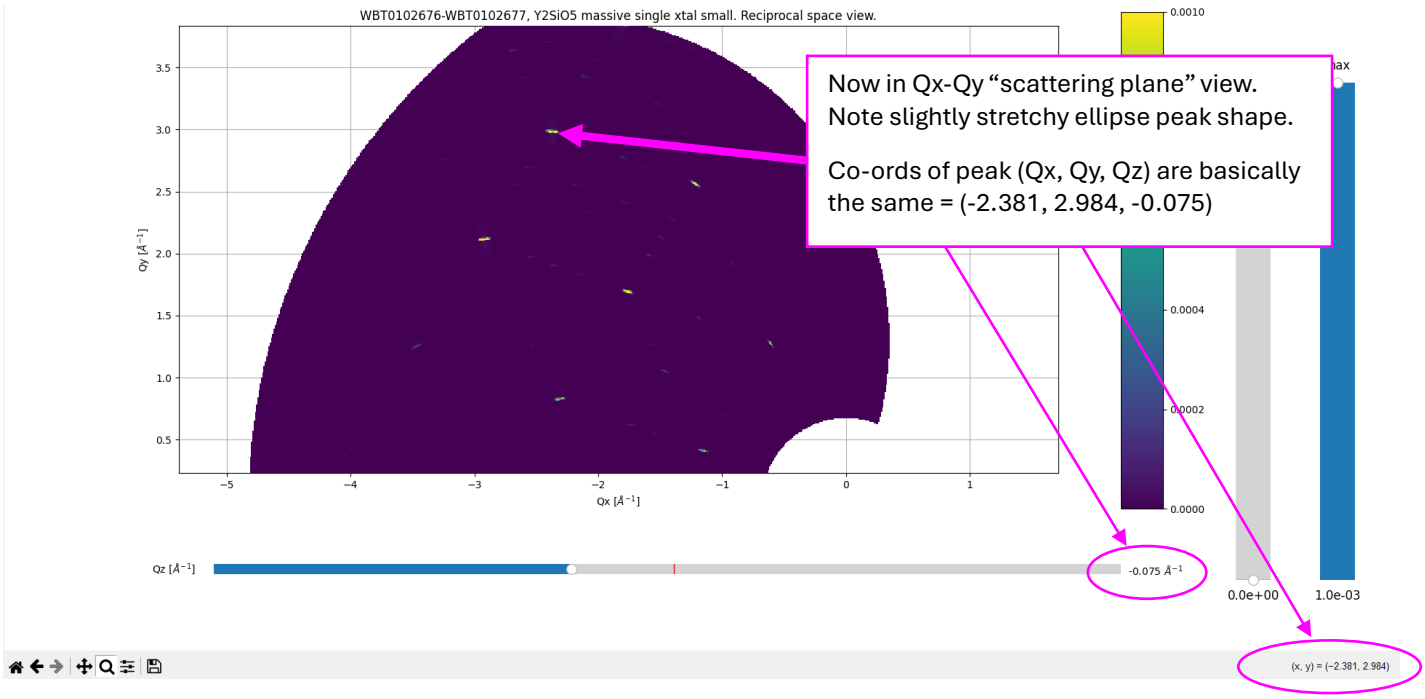
1. Used the reciprocal space viewer (Qx, Qy, Qz in inv Angstroms) script: Reciprocal_Space_View_invAA_Y2SiO5.py
2. Chose a couple of peaks from different planes and noted their coordinates. I found it easiest to do this in a plane perpendicular to the scattering plane, i.e. Qx-Qz, axis option 1: this gives you best Qz resolution (which corresponds to vertical position on the detector).
3. Used the reciprocal space conversion spreadsheet to figure out d spacing, lined up d spacings or 2θ with allowed peaks! I indexed all the peaks that I possibly could!
4. Noted the Qx, Qy, Qz and corresponding hypothesised *h, k, l*
5. Put these into the script.

There is an automatic peak search algorithm described in the DMCpy arXiv paper but I haven't investigated it.

Worked example script: UB_Matrix_and_calc_hkl_angles_Y2SiO5.py

Relevant DMCpy tutorial: *https://dmcpy.readthedocs.io/en/latest/Tutorials/View3D/Alignment.html*





	A	B	C	D	E	F	G	H	I	J
1	Handy dandy reciprocal space conversion thing					wavelength (AA)				
2						2.41				
3										
4	Qx (AA^-1)	Qy (AA^-1)	Qz (AA^-1)	Q (AA^-1)	d (AA)	2θ (deg)	Notes	h	k	l
5	-0.61	1.2776	-0.037	1.41623824	4.43653131	31.5200192		0	0	2
6	-1.714	-0.4618	-0.069	1.77646172	3.53691005	39.8381849		4	0	-2
7	-0.052	2.1373	-0.015	2.1379851	2.93883494	48.4129903		2	0	2
8	-1.752	1.6875	-0.064	2.43336316	2.5820993	55.6372985		2	0	-4
9	-2.315	0.8278	-0.08	2.45985322	2.55429278	56.2965358		4	0	-4
10	-1.21	2.5572	-0.037	2.82926507	2.22078354	65.7217913		0	0	4
11	-2.861	-0.032	-0.107	2.863179	2.19447869	66.6113164		6	0	-4
12	-2.81	-2.1813	-0.12	3.55929343	1.76529006	86.0954588		8	0	-2
13	-2.926	2.1173	-0.09	3.61282926	1.73913154	87.7161765		4	0	-6
14	-3.462	1.2576	-0.123	3.68539425	1.70488824	89.9488689		6	0	-6
15	-2.379	2.987	-0.074	3.81933057	1.6451012	94.1890308		2	0	-6
16	-4.037	0.3779	-0.137	4.05696271	1.5487412	102.164985		8	0	-6
17	-1.832	3.8568	-0.053	4.27012169	1.47143004	109.955896		0	0	6
18	-3.971	-1.7614	-0.144	4.34650514	1.44557181	112.936569		4	0	4
19				0	#DIV/0!	#DIV/0!				

Not a good idea to use two peaks on the same line, or peaks with more than one zero, so chose (4, 0, -2) and (4, 0, 4).

Put all other prominent in-plane peaks in there too, although UB calculation can only use two peaks + unit cell as input

For the above example, running the script prints the UB matrix:

[[-0.2742902 0.04330787 0.30766908]

 [-0.43633373 -0.02138978 -0.64196967]

 [-0.00272973 -0.93263608 0.02901033]]

In the same script you can provide a list of reflections and the UB matrix will be used to calculate *where* you will find those reflections. Output is saved to a .xlsx file. A3 = sample rotation angle (for this example, A3 = euler_psi), z = vertical position on detector (relative to centre of detector).

h	k	l	A3 (deg)	A4 (deg)	z (m)	two theta (deg)
1	1	0	111.1342	-11.2857	-0.77852	11.28567
2	0	2	63.78347	-48.887	0.01773	48.88702
2	0	-4	-164.505	-56.2389	-0.03596	56.23885
0	0	-4	-148.707	-66.2009	-0.02965	66.20094
6	0	-4	145.516	-66.9713	-0.03349	66.97129
4	0	2	63.00372	-72.0676	0.01118	72.06757
2	0	-6	-175.933	-94.2826	-0.03417	94.28259
0	0	6	9.39244	-110.002	0.02965	110.00223
4	0	4	32.37484	-111.704	0.01773	111.70427
10	0	-4	98.93872	-113.491	-0.02392	113.49097
4	0	6			0.02103	

Reciprocal space view in r.l.u.

Once you have a UB matrix (the example script for this section also does a quick UB matrix calculation) you can plot reciprocal space in reciprocal lattice units, and nominate your own vectors to define the projection you want to see. The remaining vector p3 = p1 x p2 is toggled via the slider.

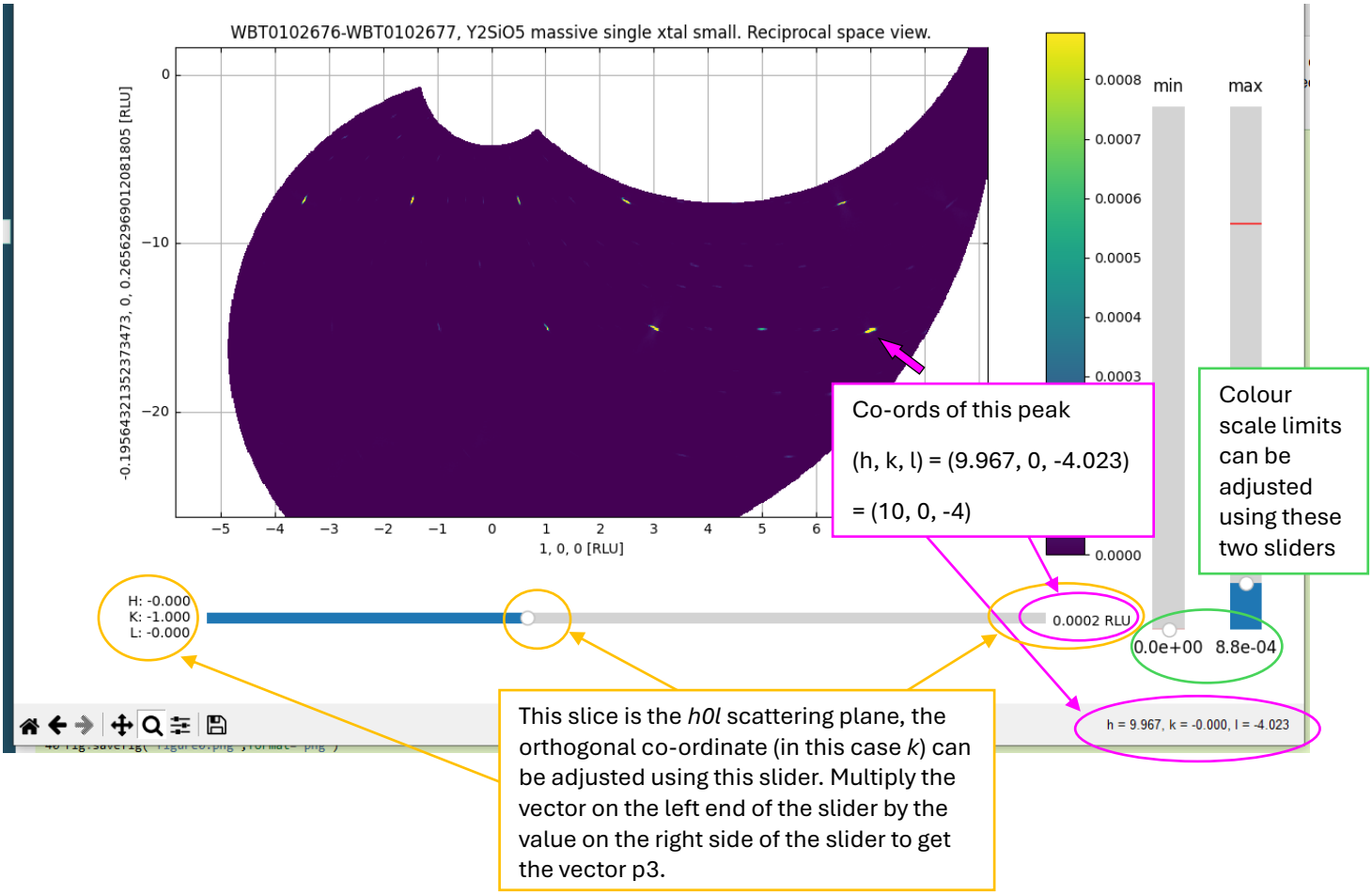
e.g. p1 = [1, 0, 0] and p2 = [0, 0, 1] gives you the *h0l* plane! And p3 = [0, 1, 0] = *k* will be controlled by the slider (p3 = multiply vector on the left end of the slider by the value in r.l.u. on the right side of the slider).

For non-orthogonal unit cells, you can't read the co-ordinates straight off the axes very easily, instead get the co-ordinates, multiply by the vector on the axis label. Then add both of these vectors together (Horace takes the same approach). **Happily for you the *h, k, l* of where your cursor also gets printed in the bottom right corner of the interactive window as shown below.**

Script: Reciprocal_Space_View_3Dalign_Y2SiO5.py

Relevant DMCpy tutorials: https://dmcpy.readthedocs.io/en/latest/Tutorials/View3D/Viewer3D_align.html

https://dmcpy.readthedocs.io/en/latest/Tutorials/View3D/Viewer3D_align_projectionVectors.html



Next steps

- At present Viewer3D doesn't offer any control over the binning / integration of the data in order to speed up the calculation. So I would aim to get DMCpy functions cut2D and cut1D working (which bin the data in a different, more accurate way, which is much slower, and currently causes my laptop to run out of RAM). Cut2D also gives nice gridlines for non-orthogonal reciprocal space axes. https://dmcpy.readthedocs.io/en/latest/Tutorials/Cut2D_hk0.html
- Test on a dataset where sample is rotated about msom
- Find more UB matrices / test on more samples!
- Eulerian cradle angles -> UB matrix -> calculate access to other scattering planes (I'll leave combining data from misc Eulerian cradle angles to Int3D)